

Actividad Evaluable 1

2025-12-15

Actividad Evaluable 1

1. Data Science

Pregunta 1:

De las siguientes preguntas, clasifica cada una como descriptiva, exploratoria, inferencia, predictiva o causal, y razona brevemente (una frase) el porqué:

1. Dado un registro de vehículos que circulan por una autopista, disponemos de su marca y modelo, país de matriculación, y tipo de vehículo (por número de ruedas). Con tal de ajustar precios de los peajes, ¿Cuántos vehículos tenemos por tipo? ¿Cuál es el tipo más frecuente? ¿De qué países tenemos más vehículos?

Podemos clasificar las tres preguntas como descriptivas, ya que en todos los casos se cuantifican los datos disponibles, sin analizar relaciones entre variables ni realizar predicciones o inferencias causales.

2. Dado un registro de visualizaciones de un servicio de video-on-demand, donde disponemos de los datos del usuario, de la película seleccionada, fecha de visualización y categoría de la película, queremos saber ¿Hay alguna preferencia en cuanto a género literario según los usuarios y su rango de edad?

Podemos clasificar esta pregunta como exploratoria porque pretende identificar la existencia de patrones o relaciones entre variables, en este caso, edad y género literario, sin partir de una hipótesis previa ni realizar inferencias.

3. Dado un registro de peticiones a un sitio web, vemos que las peticiones que provienen de una red de telefonía concreta acostumbran a ser incorrectas y provocarnos errores de servicio. ¿Podemos determinar si en el futuro, los próximos mensajes de esa red seguirán dando problemas? ¿Hemos notado el mismo efecto en otras redes de telefonía?

- Podemos clasificar la pregunta “¿Podemos determinar si en el futuro, los próximos mensajes de esa red seguirán dando problemas?” como predictiva porque busca anticipar el comportamiento futuro de las peticiones de esa red, en otras palabras, predecir resultados futuros a partir de datos históricos.
- Podemos clasificar la pregunta “¿Hemos notado el mismo efecto en otras redes de telefonía?” como exploratoria, en tanto que pretende analizar si el patrón observado se reproduce en otros subconjuntos de datos, de manera neutral objetiva sin establecer relaciones causales ni realizar predicciones.

4. Dado los registros de usuarios de un servicio de compras por internet, los usuarios pueden agruparse por preferencias de productos comprados. Queremos saber si ¿Es posible que, dado un usuario al azar y según su historial, pueda ser directamente asignado a uno o diversos grupos?

Podemos clasificar la pregunta como de inferencia porque busca asignar un usuario a uno o varios grupos definidos por preferencias de productos, a partir de su historial de compras.

Pregunta 2

Considera el siguiente escenario: Sabemos que un usuario de nuestra red empresarial ha estado usando esta para fines no relacionados con el trabajo, como por ejemplo tener un servicio web no autorizado abierto a la red (otros usuarios tienen servicios web activados y autorizados). No queremos tener que rastrear los puertos de cada PC, y sabemos que la actividad puede haber cesado. Pero podemos acceder a los registros de conexiones TCP de cada máquina de cada trabajador (hacia donde abre conexión un PC concreto). Sabemos que nuestros clientes se conectan desde lugares remotos de forma legítima, como parte de nuestro negocio, y que un trabajador puede haber habilitado temporalmente servicios de prueba. Nuestro objetivo es reducir lo posible la lista de posibles culpables, con tal de explicarles que por favor no expongan nuestros sistemas sin permiso de los operadores o la dirección. Explica con detalle cómo se podría proceder al análisis y resolución del problema mediante Data Science, indicando de donde se obtendrían los datos, qué tratamiento deberían recibir, qué preguntas hacerse para resolver el problema, qué datos y gráficos se obtendrían, y cómo se comunicarían estos.

El problema planteado consiste en identificar posibles usuarios internos que hayan expuesto servicios web no autorizados utilizando unicamente registros de conexiones TCP salientes, evitando tecnicas intrusivas como el escaneo de puertos y teniendo en cuenta que la actividad puede haber sido temporal o ya finalizada.

El primer paso seria la obtencion de datos. En este caso, la fuente principal serian los registros de conexiones TCP de los equipos de los trabajadores, donde se recojan al menos la IP origen interna, IP destino, puerto destino, timestamp y si es posible, volumen de datos transmitidos. Estos registros podrian obtenerse de firewalls, sistemas de monitorizacion de red, logs de proxies o herramientas de NetFlow.

Una vez recopilados los datos, seria necesario realizar un proceso de limpieza y normalizacion. Esto incluiria la eliminacion de registros incompletos o corruptos, la unificacion de formatos de fecha y hora, y la anonimizacion de datos sensibles para cumplir con principios de privacidad. Tambien seria importante etiquetar o filtrar conexiones legitimas conocidas, como accesos a servicios corporativos, conexiones VPN de clientes o servidores externos autorizados.

El siguiente paso seria formular preguntas clave del analisis, que guiaran la exploracion de los datos. Algunas preguntas serian: Existen equipos internos que realicen conexiones entrantes recurrentes desde multiples IPs externas? Se observan patrones de conexion tipicos de servicios web? Hay equipos cuyo patron de conexiones difiere significativamente del comportamiento normal del resto de usuarios? Estas conexiones se concentran en determinadas franjas horarias o periodos concretos?

A nivel de analisis y visualizacion, se podran generar graficos de distribucion del numero de conexiones entrantes por equipo, series temporales de conexiones TCP por host interno y mapas de calor que relacionen equipos internos con puertos y frecuencias de conexion. Tambien serian utiles tecnicas de deteccion de anomalias para identificar equipos cuyo comportamiento se aleje del patron general de la red. El resultado esperado no seria una acusacion directa, sino una reduccion progresiva del conjunto de sospechosos, identificando aquellos equipos que presentan patrones compatibles con la exposicion temporal de servicios web no autorizados. Esta lista permitiria una intervencion manual posterior.

Finalmente, los resultados se comunicarian de forma clara y comprensible, combinando graficos visuales con un resumen explicativos que justifique por que ciertos equipos han sido destacados. El objetivo de la comunicacion no seria sancionador, sino preventivo y educativo, facilitando el dialogo con los usuarios implicados para explicar los riesgos de exponer servicios sin autorizacion y reforzar las politicas de seguridad de la organizacion.

2. Introducción a R

Preparación de los datos

Para poder analizar los datos, primero debemos cargarlos dentro de R. Para ello, vamos a usar la libreria readr, que nos permitira realizar dicha tarea.

```
#instrucción para descargar la libreria de readr. En caso de ya tenerla instalada, se puede comentar
#install.packages("readr")
library("readr")

logs <- read_log("epa-http.csv")
```

```
##
## -- Column specification -----
## cols(
##   X1 = col_character(),
##   X2 = col_character(),
##   X3 = col_character(),
##   X4 = col_double(),
##   X5 = col_double()
## )

## Warning: 21 parsing failures.
## row col expected actual file
## 7527 -- 5 columns 4 columns 'epa-http.csv'
## 7528 -- 5 columns 4 columns 'epa-http.csv'
## 7529 -- 5 columns 4 columns 'epa-http.csv'
## 7549 -- 5 columns 4 columns 'epa-http.csv'
## 7550 -- 5 columns 4 columns 'epa-http.csv'
## ....
## See problems(...) for more details.
```

Si nos fijamos en la salida anterior, observamos 2 datos importantes respecto a al almacenamiento de datos anterior. Primero nos aparecen las columnas que tiene nuestro dataset, con su respectivo tipo cada una i seguido tenemos un conjunto de Warnings para algunos de nuestros elementos.

Empezaremos analizando que es lo que sucede con estos Warnings, por lo que empezaremos gurdando todos los id de las filas donde ha sucedido algún error.

```
errores <- problems(logs)$row
print(errores)
```

```
## [1] 7527 7528 7529 7549 7550 7551 7622 7623 7624 7655 7657 7658 7699 7701 7702
## [16] 7759 7761 7763 7781 7783 7786
```

Seguidamente, para todos las filas donde hay errores, obtenemos todas las columnas.

```
filas_con_errores <- logs[errores, ]
print(filas_con_errores)
```

```
## # A tibble: 21 x 5
##   X1          X2          X3          X4    X5
##   <chr>      <chr>      <chr>      <dbl> <dbl>
## 1 bastion.fdic.gov 30:09:36:04 GET /enviro/gif/blueball.gifalt= HT~ 404    NA
## 2 bastion.fdic.gov 30:09:36:04 GET /enviro/gif/tris.gifalt= HTTP/1~ 404    NA
## 3 bastion.fdic.gov 30:09:36:04 GET /enviro/gif/efacts.gifalt= HTTP~ 404    NA
## 4 bastion.fdic.gov 30:09:36:25 GET /enviro/gif/efacts.gifalt= HTTP~ 404    NA
```

```
## 5 bastion.fdic.gov 30:09:36:25 GET /enviro/gif/blueball.gifalt= HT~ 404 NA
## 6 bastion.fdic.gov 30:09:36:26 GET /enviro/gif/tris.gifalt= HTTP/1~ 404 NA
## 7 bastion.fdic.gov 30:09:37:21 GET /enviro/gif/blueball.gifalt= HT~ 404 NA
## 8 bastion.fdic.gov 30:09:37:21 GET /enviro/gif/tris.gifalt= HTTP/1~ 404 NA
## 9 bastion.fdic.gov 30:09:37:22 GET /enviro/gif/efacts.gifalt= HTTP~ 404 NA
## 10 bastion.fdic.gov 30:09:37:51 GET /enviro/gif/efacts.gifalt= HTTP~ 404 NA
## # i 11 more rows
```

Si obserbamos los datos obtenidos, la columna X4, que corresponde al estado de las llamadas HTTP, siempre da como resultado 404 (Not Found). Como hemos visto antes, la columna X5 es representada por col_double (que son numeros) por lo que si R detecta otro tipo de dato que no sea este, por defecto lo pondra como NA. Para poder tenerlos en cuenta, hemos decidido modificar los datos obtenidos por tal de hacer lo siguiente: Cualquier elemento existente dentro de la columna X5 con valor NA, será sustituido por el valor 0.

```
#install.packages("dplyr")
#install.packages("tidyr")
library("dplyr")
```

```
##
## S'està adjuntant el paquet: 'dplyr'
```

```
## Els següents objectes estan emmascarats des de 'package:stats':
##
## filter, lag
```

```
## Els següents objectes estan emmascarats des de 'package:base':
##
## intersect, setdiff, setequal, union
```

```
library("tidyr")

logs <- logs %>% mutate(X5 = replace_na(X5, 0))
print(logs)
```

```
## # A tibble: 47,748 x 5
##   X1                X2                X3                X4    X5
##   <chr>            <chr>            <chr>            <dbl> <dbl>
## 1 141.243.1.172    29:23:53:25 GET /Software.html HTTP/1.0    200 1497
## 2 query2.lycos.cs.cmu.edu 29:23:53:36 GET /Consumer.html HTTP/1.0    200 1325
## 3 tanuki.twics.com 29:23:53:53 GET /News.html HTTP/1.0      200 1014
## 4 wpbf12-45.gate.net 29:23:54:15 GET / HTTP/1.0              200 4889
## 5 wpbf12-45.gate.net 29:23:54:16 GET /icons/circle_logo_small~ 200 2624
## 6 wpbf12-45.gate.net 29:23:54:18 GET /logos/small_gopher.gif ~ 200 935
## 7 140.112.68.165    29:23:54:19 GET /logos/us-flag.gif HTTP/~ 200 2788
## 8 wpbf12-45.gate.net 29:23:54:19 GET /logos/small_ftp.gif HTT~ 200 124
## 9 wpbf12-45.gate.net 29:23:54:19 GET /icons/book.gif HTTP/1.0   200 156
## 10 wpbf12-45.gate.net 29:23:54:19 GET /logos/us-flag.gif HTTP/~ 200 2788
## # i 47,738 more rows
```

Como para alguna pregunta mas adelante, hara analizar datos por la fecha, procedemos a parsear los datos con strptime(), que transforma los los datos siguiendo un patrón en una fecha como tal.

```
logs$X2 <- strptime(logs$X2, format = "%d:%H:%M:%S")
```

Ahora que el dataset esta limpio, procedemos a poner los nombres de las columnas.

```
colnames(logs) <- c("IP", "Timestamp", "Petición", "CodigoRespuesta", "BytesReply")
```

```
head(logs)
```

```
## # A tibble: 6 x 5
##   IP                Timestamp      Petición CodigoRespuesta BytesReply
##   <chr>            <dtm>         <chr>         <dbl>         <dbl>
## 1 141.243.1.172    2025-12-29 23:53:25 GET /So~         200         1497
## 2 query2.lycos.cs.cmu.e~ 2025-12-29 23:53:36 GET /Co~         200         1325
## 3 tanuki.twics.com    2025-12-29 23:53:53 GET /Ne~         200         1014
## 4 wpbfl2-45.gate.net  2025-12-29 23:54:15 GET / H~         200         4889
## 5 wpbfl2-45.gate.net  2025-12-29 23:54:16 GET /ic~         200         2624
## 6 wpbfl2-45.gate.net  2025-12-29 23:54:18 GET /lo~         200          935
```

Pregunta 1

1. Cuales son las dimensiones del dataset cargado (número de filas y columnas)

```
rows <- nrow(logs)
columns <- ncol(logs)
```

```
print(rows)
```

```
## [1] 47748
```

```
print(columns)
```

```
## [1] 5
```

2. Valor medio de la columna Bytes

```
media <- mean(logs$BytesReply)
```

```
print(media)
```

```
## [1] 6531.457
```

Pregunta 2

De las diferentes IPs de origen accediendo al servidor, ¿cuántas pertenecen a una IP claramente educativa (que contenga “.edu”)?

```
#install.packages("stringr")
library("stringr")
```

```
#Buscamos los logs que contengan .edu y los almacenamos en una nueva variable
ips_edu <- str_subset(logs$IP, "\\..edu$")
head(ips_edu)
```

```
## [1] "query2.lycos.cs.cmu.edu" "flaxman-q950.uoregon.edu"
## [3] "flaxman-q950.uoregon.edu" "flaxman-q950.uoregon.edu"
## [5] "flaxman-q950.uoregon.edu" "flaxman-q950.uoregon.edu"
```

Primero realizaremos el calculo total de IPs educativas que aparecen en los logs.

```
total_ips_edu <- length(ips_edu)

print(total_ips_edu)
```

```
## [1] 6317
```

Seguidamente, con la funcion unique(), podemos obtener todas las IPs distintas que aparecen.

```
ips_edu_unicas <- unique(ips_edu)
total_ips_edu_unicas <- length(ips_edu_unicas)

print(total_ips_edu_unicas)
```

```
## [1] 361
```

Pregunta 3

De todas las peticiones recibidas por el servidor cual es la hora en la que hay mayor volumen de peticiones HTTP de tipo “GET”?

```
#install.packages("lubridate")
library("lubridate")
```

```
##
## S'està adjuntant el paquet: 'lubridate'
```

```
## Els següents objectes estan emmascarats des de 'package:base':
##
##     date, intersect, setdiff, union
```

```
#Obtenemos todos los logs que realizen una peticion GET
peticiones_get <- logs %>%
  filter(str_detect(Peticion, "^GET"))

#Calculamos la hora con mas peticiones GET
hora_pico <- peticiones_get %>%
  mutate(
    #Como tenemos los datos tratados con anterioridad, podemos obtener el dia y la hora de un timestamp
    dia = day(peticiones_get$Timestamp),
    hora = hour(peticiones_get$Timestamp)
  ) %>%
#Hacemos el conteo de gets por cada dia y hora
  count(dia, hora, sort = TRUE)
```

La hora donde hay mas logs de con peticiones GET es:

```
print(hora_pico[1,])
```

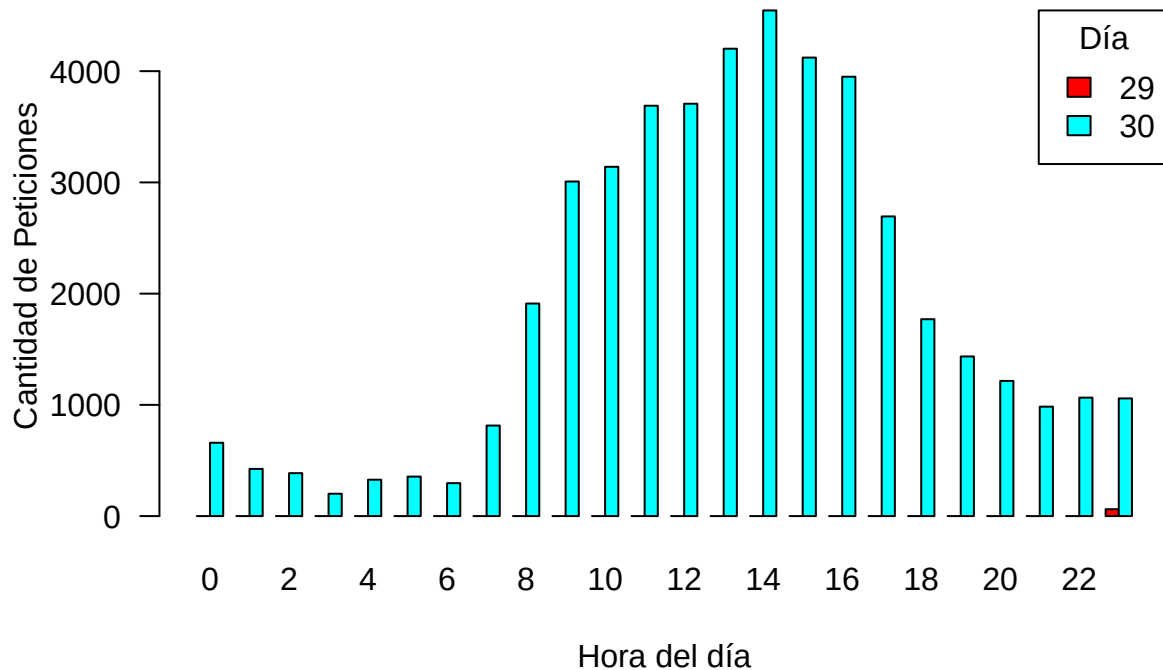
```
## # A tibble: 1 x 3  
##   dia hora    n  
##   <int> <int> <int>  
## 1    30    14 4546
```

Finalmente podemos observar los resultados obtenidos en forma de tabla.

Esta primera tabla es usando el barplot simple de R:

```
#Juntamos dia y hora en una misma variable  
datos_tabla <- xtabs(n ~ dia + hora, data = hora_pico)  
  
barplot(  
  datos_tabla,  
  beside = TRUE,  
  col = rainbow(nrow(datos_tabla)),  
  legend = rownames(datos_tabla),  
  args.legend = list(x = "topright", title = "Día"),  
  
  # Etiquetas  
  main = "Peticiones GET por Hora y Día",  
  xlab = "Hora del día",  
  ylab = "Cantidad de Peticiones",  
  las = 1  
)
```

Peticiones GET por Hora y Día



Esta segunda tabla es un poco mas completa, pero mas compleja de configurar. Esta usa la libreria ggplot2 y crea un plot igual al anterior, pero con mayor calidad y con una mejora estetica.

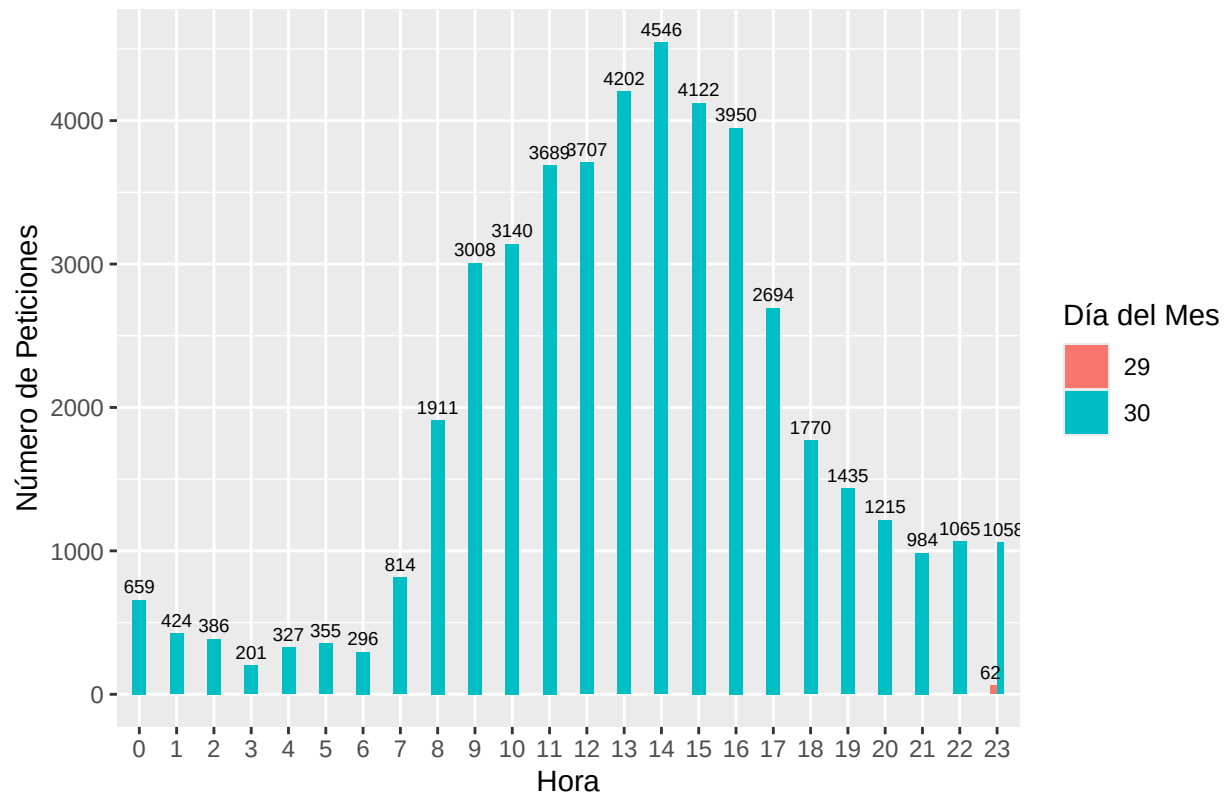
```
#install.packages("ggplot2")
library(ggplot2)

#Codigo para crear un
ggplot(hora_pico, aes(x = factor(hora), y = n, fill = factor(dia))) +
  geom_col(position = "dodge", width = 0.35) +

#Codigo para añadir el valor exacto encima de cada barra
geom_text(aes(label = n),
           position = position_dodge(width = 0.7),
           vjust = -0.5, # Mueve el texto un poco hacia arriba
           size = 2.5) + # Tamaño del texto

#Etiquetas y estética
labs(
  title = "Peticiones GET por Día y Hora",
  x = "Hora",
  y = "Número de Peticiones",
  fill = "Día del Mes"
)
```


Peticiones GET por Día y Hora



Pregunta 4

De las peticiones hechas por instituciones educativas (.edu), ¿Cuántos bytes en total se han transmitido, en peticiones de descarga de ficheros de texto “.txt”?

```
logs2 <- logs %>%
  mutate(
    req_parts = str_split(logs$Petición, " ", simplify = TRUE),
    method = req_parts[,1],
    url = req_parts[,2],
    proto = req_parts[,3]
  )

p4 <- logs2 %>%
  filter(str_detect(IP, "\\..edu")) %>%
  filter(str_detect(url, "\\..txt$")) %>%
  summarise(total_bytes_txt = sum(BytesReply))

print(p4)
```

```
## # A tibble: 1 x 1
##   total_bytes_txt
##           <dbl>
## 1           106806
```

```
print(p4$total_bytes_txt)
```

```
## [1] 106806
```

Para responder a esta pregunta se filtraron las peticiones cuyo host pertenece a instituciones educativas (dominios que contienen “.edu”) y cuya URL corresponde a descargas de ficheros de texto con extension “.txt”. Posteriormente, se calcularon los bytes totales transmitidos sumando la columna ‘bytes’, tras convertirla previamente a formato numerico y tratar valores ausentes.

Pregunta 5

Si separamos la petición en 3 partes (Tipo, URL, Protocolo), usando `str_split` y el separador “ ” (espacio), ¿cuántas peticiones buscan directamente la URL = “/”?

```
#Solo ejecutar la primera linea la primera vez, luego hay que comentarla a no ser que se limpien los da  
logs$Peticon <- str_split(logs$Peticon, " ", simplify = TRUE)  
sum(logs$Peticon[,2] == "/")
```

```
## [1] 2382
```

Pregunta 6

Aprovechando que hemos separado la petición en 3 partes (Tipo, URL, Protocolo) ¿Cuántas peticiones NO tienen como protocolo “HTTP/0.2”?

```
sum(logs$Peticon[,3] != "HTTP/0.2")
```

```
## [1] 47747
```

```
http02 <- logs %>% filter(str_detect(logs$Peticon[,3], "HTTP/0.2"))  
head(http02)
```

```
## # A tibble: 1 x 5  
##   IP          Timestamp      Peticon[,1]  CodigoRespuesta BytesReply  
##   <chr>        <dtm>         <chr>          <dbl>         <dbl>  
## 1 oa8.r5ora.epa.gov 2025-12-30 09:36:49 GET           404           0  
## # i 1 more variable: Peticon[2:3] <chr>
```